

Sujet d'examen UE NFE204 - Session FOAD

Année universitaire 2018-2019

Examen première session : 21 juin 2019

Consignes

Durée de l'examen : 3h00 ; Documents non autorisés ; Calculatrice autorisée

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 5 pages.

Important : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

L'ensemble de l'examen doit vous permettre de montrer que vous avez assimilé les principales notions vues en cours et en TP et que vous savez les mettre en application et les expliquer. La clarté, la concision, la précision des réponses sera donc appréciée.

Première partie : modélisation par documents structurés (7 pts)

Le Grand Débat a permis de collecter les contributions de beaucoup de Français. Le gouvernement a défini 5 sujets ("Impôts", "Transition écologique", etc.) et proposé un ensemble de questions pour chaque sujet (ex : "Payez-vous trop d'impôts" ou "Faut-il arrêter d'importer du pétrole"). Chaque citoyen (anonymisé : on ne connaît que sa région de résidence) peut alors entrer sa contribution sous la forme d'un texte libre.

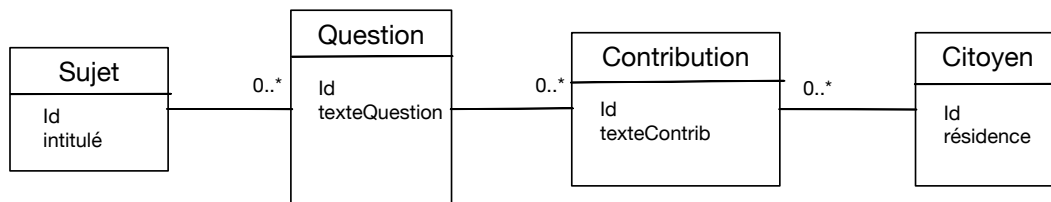


FIGURE 1 – Schéma de la base du Grand Débat

Les contributions sont stockées dans une base relationnelle dont le schéma est donné par la figure 1 en UML. Elles sont publiées ensuite sous la forme de documents JSON à des fins d'analyse.

1. Donnez le schéma relationnel correspondant à la figure 1.
2. Donnez un exemple de base relationnelle avec quelques données (très simplifiées) dans chaque table : deux sujets, deux questions (une par sujet), trois contributions par deux citoyens.
3. Donnez une représentation JSON d'un document rassemblant toutes les informations (questions, contributions, citoyens) relatives au premier sujet que vous avez choisi.
4. Indiquez le traitement MapReduce qui permet de passer de la représentation précédente à une représentation centrée sur les citoyens.

5. Vous anticipez un très grand nombre de contributions et souhaitez donc prévoir un stockage des documents JSON dans un système distribué. Quelle représentation vous semble préférable parmi les deux précédentes pour tirer au mieux parti de la distribution ?
6. Vous développez un programme `ANALYSECONTRIB(d)` qui s'applique à un document *d* contenant l'ensemble des contributions relatives à une question. Quel format ce document devrait-il avoir à votre avis pour atteindre une scalabilité maximale ? Quelle chaîne de traitement permettrait obtenir un tel document à partir de l'une des représentations JSON déjà proposées (au choix) ?

Solution :

1. Le schéma de la base
 - (a) Sujet (**id**, intitulé)
 - (b) Question (**id**, texte, idSujet)
 - (c) Citoyen (**id**, résidence)
 - (d) Contribution (**id**, texteContrib, idCitoyen, idQuestion)
2. Donnez un exemple de base relationnelle avec quelques données (très simplifiées) dans chaque table : deux sujets, deux questions (une par sujet), trois contributions par deux citoyens.

id	nom	id	nom	idSujet
20	Impôts	100	Payez-vous trop d'impôts	20
21	Transition écologique	101	Faut-il arrêter le pétrole	21

La table *Sujet*

La table *Question*

id	Résidence
chocho@monsite.com	Paris
alain@dugenou.com	Marseille

La table *Citoyen*

id	texteContrib	idCitoyen	idQuestion
1	100	chocho@monsite.com	C'est insupportable
2	100	alain@dugenou.com	Moi ça me va
3	101	chocho@monsite.com	Surtout pas
4	101	alain@dugenou.com	Bien sûr

La table *Contribution*

```

3. {
  {
    "_id": 978,
    "sujet": "Impôts",
    "questions": [
      {
        "question": "Payez-vous trop d'impôts?",
        "contributions": [
          {
            "citoyen": "chocho@monsite.com",
            "texteContrib": "C'est insupportable"
          },
          {
            "citoyen": "alain@dugenou.com",
            "texteContrib": "Moi ça me va"
          }
        ]
      }
    ]
  },
  {
    "question": "Faut-il arrêter le pétrole",
    "contributions": [
      {
        "citoyen": "chocho@monsite.com",
        "texteContrib": "Surtout pas"
      }
    ]
  }
}

```

```

    {"citoyen": "d@alta.com",
      "texteContrib": "Bien sûr"}
  ]
}

```

- La fonction de Map prend en entrée un document "sujet", effectue une boucle sur les questions puis sur les contributions, et émet des paires avec comme clé le citoyen, et comme valeur le sujet, la question et le texte de la contribution.
La fonction de reduce agrège toutes les contributions d'un même citoyen en un document JSON.
- Il n'y a que 5 sujets, ce qui limite les possibilités de distribution. Il faut utiliser les documents centrés sur les citoyens.
- Un MapReduce produit de documents centrés sur les questions. Même chose que ci-dessus, mais avec une boucle en moins dans la fonction de Map.

Deuxième partie : recherche d'information (8 pts)

Maintenant on veut analyser les contributions au grand débat et on commence par construire un index plein texte. On va travailler sur l'échantillon suivant des contributions $c_1, \dots, c_5$.

- c_1 Non seulement on est écrasés d'impôts, mais en plus l'essence augmente sans fin.
- c_2 C'est bien beau de payer des impôts sur le pétrole ou le logement, il faudrait que ça serve à l'écologie.
- c_3 Des impôts, encore des impôts. Stop!
- c_4 Je propose que les taxes sur l'essence financent la rénovation des logements, c'est pour l'écologie.
- c_5 Acheter de l'essence, c'est payer des impôts. J'en arrive à devoir choisir entre mon essence et mon logement.

On va se limiter dans un premier temps au vocabulaire (impôt, essence, écologie, logement). (NB : on suppose une phase préalable de normalisation qui élimine les pluriels, majuscules, etc.)

- Donnez une matrice d'incidence (ou index) contenant les tf, et un tableau donnant les idf (sans le log), pour les termes précédents. Placez les termes en ligne, les contributions en colonne.
Donnez également les normes des vecteurs représentant ces commentaires.

Solution :

	c1	c2	c3	c4	c5
impôt 5/4	1	1	2	0	1
essence 5/3	1	0	0	1	2
écologie 5/2	0	1	0	1	0
logement 5/3	0	1	0	1	1

Les normes

$$\|c_1\| = \sqrt{1+1}; \|c_2\| = \sqrt{3}; \|c_3\| = \sqrt{4}; \|c_4\| = \sqrt{3}, \|c_5\| = \sqrt{6}$$

- Dans un espace à deux dimensions avec les axes "impôt" en abscisse et "essence" en ordonnée, placer les vecteurs correspondant aux documents normalisés.

- Donner les résultats classés par similarité cosinus basée sur les tf (on ignore l'idf) pour la requête `impôt essence`. Expliquez brièvement la raison du classement.

Solution : Les cosinus (requête non normalisée).

— `impôt` et `essence`

$$c1 : \frac{1+1}{\sqrt{2}} ; c2 : \frac{1}{\sqrt{3}} ; c3 : \frac{2}{\sqrt{3}} ; c4 : \frac{1}{\sqrt{3}} ; c5 : \frac{1+2}{\sqrt{6}} ;$$

- Quelqu'un remarque que le terme "taxe" est synonyme d'"impôt" et "pétrole" d'essence. Proposez les changements dans la matrice pour tenir compte de cette synonymie. Qu'est-ce cela change pour la requête `impôt essence` ? Commentez en particulier le classement des documents `c1` et `c4`.

Solution : Un 1 remplace le 0 dans la cellule (`impôt`, `c4`), et l'idf de `impôt` devient 1 (on pourrait donc le considérer comme un mot vide).

Le document `c4` remonte en seconde position. Il reste derrière `c1` qui ne parle *que* d'`impôt` et d'`essence`.

- Proposez une formule pour mesurer la co-occurrence de deux termes, et indiquez une méthode pour calculer cet indicateur à partir de la matrice. Quelle est la paire de termes ayant la meilleure co-occurrence (en tenant compte des synonymes) ?

Solution : Pour chaque paire, on regarde les colonnes qui ne contiennent pas de 0. Les termes `impôt essence` ont la meilleure co-occurrence (3 sur 5) en tenant compte des synonymes.

- On considère que le bon résultat d'une recherche avec le terme `impôt` est celui qui tient compte des synonymes, et ramène donc les documents contenant soit `impôt`, soit `taxe`. Si on fait une recherche *booléenne* (sans classement donc), quel est le rappel et la précision pour notre requête `impôt essence` avant et après l'adaptation de la matrice aux synonymes ?

Solution : Avant : il manque `c4`. La précision est 1, le rappel 2/3. Après : la précision et le rappel sont à 1.

Troisième partie : flux (3 pts)

Le grand débat ayant lieu en continu, on décide de mettre en place une chaîne de traitement de flux qui prend en entrée un flux de contributions, nommé `fluxContrib`. Voici son code Scala, très proche de celui vu en cours.

```
case class CompteurMot(mot: String, compteur: Int)

fluxContrib.map({ str => FiltrageVocabulaire(str, V) })
  .flatMap({ str => str.split("\\W+") })
  .map({ m => CompteurMot(m, 1) })
  .keyBy("mot")
  .sum("compteur")
```

La fonction `FiltrageVocabulaire(s, V)` prend une chaîne `s` en entrée et un vocabulaire `V`, et renvoie une chaîne `s'` dans laquelle on ne garde que les mots de `s` qui appartiennent à `V`.

- Que produit cette chaîne de traitement ?

Solution : La fréquence des termes

2. Y a-t-il un degré de parallélisation maximal lié à l'un des paramètres de ce calcul ? Lequel et pourquoi ?

Solution : La parallélisation est limitée par la taille du vocabulaire

3. Comment adapter ce traitement pour prendre en compte les synonymes (vous pouvez supposer que vous disposez de toute fonction nécessaire) ?

Solution : Il faut une structure S contenant les synonymes, et une fonction `Synonymes(str, S)` qui remplace tous les mots par le premier synonyme (par exemple). La fonction s'appelle avant le `flatMap`.

Quatrième partie : question de cours (2 pts)

Dans un système de stockage distribué de type Master-Esclave, vaut-il mieux lire en passant par le maître ou en allant directement sur l'esclave qui stocke le document cherché ? Discuter des avantages et inconvénients des deux solutions.